

Hardware Implementation of FANET Using FANS

Subhodeep Choudhury*
Electrical and Electronics Engineering
Birla Institute of Technology and Science
Goa, Goa, India
f20212913@goa.bits-pilani.ac.in

Abhishek Joshi
Information and Communication Technology
Pandit Deendayal Energy University
Gandhinagar, Gujarat, India
abhishek.joshi@sot.pdpu.ac.in

Nitya Ahuja*
Mechanical Engineering
Birla Institute of Technology and Science
Goa, Goa, India
f20212293@goa.bits-pilani.ac.in

Sarang C. Dhongdi
Electrical and Electronics Engineering
Birla Institute of Technology and Science
Goa, Goa, India
sarang@goa.bits-pilani.ac.in

Abstract

Flying Ad-hoc Networks (FANETs) are considered to be ad-hoc network formed by Unmanned Aerial Vehicles (UAVs). FANETs are widely used in both military and civilian domains, like surveillance or emergency response. Due to the highly dynamic movement of UAVs in FANETs, communication protocols must be thoroughly tested before deployment. To ensure consistent performance, thorough testing must be performed in both simulated and real-world contexts. In this study, the FANS (Flying Ad-hoc Network Simulator) co-simulation platform is expanded. This platform combines ns-3 network simulation, ROS robotics software development, and Gazebo drone simulation. It is further connected to hardware emulation using an embedded system platform, which includes the NVIDIA Jetson Nano and Pixhawk-4 flight controller. The simulation framework is designed to measure important network parameters such as end-to-end delay and Packet Delivery Ratio (PDR) in a decentralized Wi-Fi ad-hoc network setup. In addition to the simulation, a hardware testbed consisting of three drones is created. In this configuration, UAVs communicate while flying in a straight line formation. The findings of this real-world implementation are contrasted with those acquired through simulation. The simulation and hardware test results are presented to show that the hardware integration was successful. This work lays the groundwork for future research and testing of various networking protocols in FANETs.

CCS Concepts

• **Networks** → **Network performance analysis**; **Network protocols**; **Mobile ad hoc networks**; • **Hardware** → **Hardware validation**.

Keywords

Flying Ad-hoc Networks, Unmanned Aerial Vehicles, Hardware Implementation, ns-3, ROS

*These authors have contributed equally to this work and share first authorship.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICNS3 2025, Osaka, Japan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1517-4/25/08
<https://doi.org/10.1145/3747204.3747223>

ACM Reference Format:

Subhodeep Choudhury, Nitya Ahuja, Abhishek Joshi, and Sarang C. Dhongdi. 2025. Hardware Implementation of FANET Using FANS. In *2025 International Conference on ns-3 (ICNS3 2025)*, August 19–21, 2025, Osaka, Japan. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3747204.3747223>

1 Introduction

Flying Ad hoc Networks (FANETs) are a subset of Wireless Sensor Networks (WSNs), characterized by specific constraints such as node mobility, sparse node deployment, and frequent changes in topology. FANETs are becoming increasingly popular in various application areas, including search and rescue operations, environmental sensing, traffic and urban monitoring, and disaster area monitoring, among others [4, 7].

Conventional network simulators (e.g., ns-3) effectively model network-layer behavior (routing, MAC protocols) but lack realistic UAV dynamics and environmental interactions. On the contrary, robotic simulators like Gazebo integrated with Robot Operating System (ROS) represent physical and kinematic constraints in an accurate manner but do not include message exchanges which are essential for building network protocol stacks. To overcome limitations imposed by UAV simulators and network simulators, a co-simulator platform needs to be considered which combines features of network simulators and robotic simulators.

Prior research in [3] developed FANS (Flying Ad-hoc Network Simulator), a co-simulation platform integrating ns-3 [5], Gazebo [6], and ROS [8] to jointly model UAV dynamics and network protocols. FANS supports simulation of ad-hoc Wi-Fi communication, swarm collision avoidance, and dynamic channel conditions – distinguishing it from infrastructure-mode tools like FlyNetSim [2] and CORNET [1]. While FANS effectively analyzes network performance metrics (end-to-end delay, packet delivery ratio), its dependence on ideal wireless channel models and abstracted hardware behavior raises questions regarding real-world applicability.

To address this limitation, we extend FANS through hardware implementation. The simulation framework is deployed on embedded platforms using NVIDIA Jetson Nano boards for onboard computation paired with Pixhawk-4 flight controllers. This hardware testbed evaluates FANET protocols under realistic conditions, revealing discrepancies between simulation and hardware results. For example, hardware experiments expose latency spikes from



Figure 1: UAV Hardware Testbed for FANS Extended

wireless interference, kernel-level socket handling, and buffer bloat – phenomena absent in systematic packet forwarding simulations.

This research employs an integrated methodology combining simulations with hardware emulations to study FANETs under real-world constraints. The simulation component utilizes ns-3 to model ad-hoc Wi-Fi communication in three-node UAV swarms, incorporating modules for medium access control, packet forwarding/routing, and application-layer data/control message transmission. Further, this simulated environment is transitioned to a physical testbed where Jetson Nano manages real-time UDP communication, ROS provides UAV state estimation, and Pixhawk executes low-level flight control. Comparative analysis of simulation and hardware results quantifies the impact of real-world factors (channel contention, clock drift) on network performance (end-to-end delay, PDR). This approach validates FANS on embedded hardware while highlighting simulation-to-reality challenges: synchronization issues between ROS and ns-3, buffer limitations, and the critical unsolved problem of precise multi-node time synchronization in real-world FANET deployments – a factor directly impacting protocol performance and network reliability.

2 Literature Survey

The increasing amount of interest in Flying Ad-hoc Networks (FANETs) has encouraged the development of simulation and co-simulation platforms which deal with specific challenges encountered by UAV swarms. These platforms integrate network simulations with robotic dynamics to provide a comprehensive evaluation of FANET protocols. Key advances in this field are compared in the following discussion, taking into account their strengths, limits, and significance to this work. The highlights of the comparative study are summarized in Table 1.

FlyNetSim [2] combines the ns-3 network simulator with the ArduPilot autopilot software. It uses a publish-subscribe middleware to synchronize flight dynamics and network protocols. This platform supports infrastructure-mode WiFi, which enables UAVs to interact via centralized access points. While this configuration simplifies routing, it also introduces single points of failure, which

are unsuitable for decentralized swarm applications. Furthermore, the access point-dependent architecture of FlyNetSim restricts scalability and robustness in dynamic contexts.

CORNET (Co-Simulation Middleware for Robot Networks) [1] uses Gazebo/ROS for UAV dynamics and ns-3 for network simulation. Similar to FlyNetSim, CORNET uses infrastructure-mode Wi-Fi to mimic Cyber-Physical Systems in multi-UAV situations. The platform adequately captures physical interactions, but it does not enable the ad-hoc routing protocols required for decentralized FANETs. The architecture of CORNET also limits hardware validation possibilities, resulting in gaps between simulation and deployment.

FANS (Flying Ad-hoc Network Simulator) [3] expands on FlyNetSim and CORNET with native ad-hoc Wi-Fi support. Unlike infrastructure-dependent alternatives, FANS supports direct node-to-node communication in accordance with decentralized requirements of FANET. The platform combines ns-3, Gazebo, and ROS to simulate both UAV dynamics and network protocols. Gazebo provides realistic mobility, whereas ns-3 performs ad-hoc routing under changing channel conditions, facilitating applications such as swarm collision avoidance and adaptive routing.

The current FANS extension includes hardware emulation for embedded devices. FANET network protocols can be tested in physical environments using Jetson Nano boards and Pixhawk 4 flying controls. This implementation reveals inconsistencies between simulation and hardware results, including greater delays and reduced packet delivery ratios caused by wireless interference, buffer bloat, and OS jitter.

In **FANS Extended**, the platform is extended to handle real-world UAV communication via UDP-based packet forwarding. The hardware implementation uses `EmuFdNetDeviceHelper` in ns-3 to tie simulated nodes to physical interfaces (such as `wlan0`), allowing for real-time inter-drone communication. Key components are:

Sender Node: Employs raw sockets to transmit UDP packets (1000B, 1 pkt/s) via `GenerateTraffic()`. ROS detects flight mode (`mavros_msgs/State`) and initiates broadcasts.

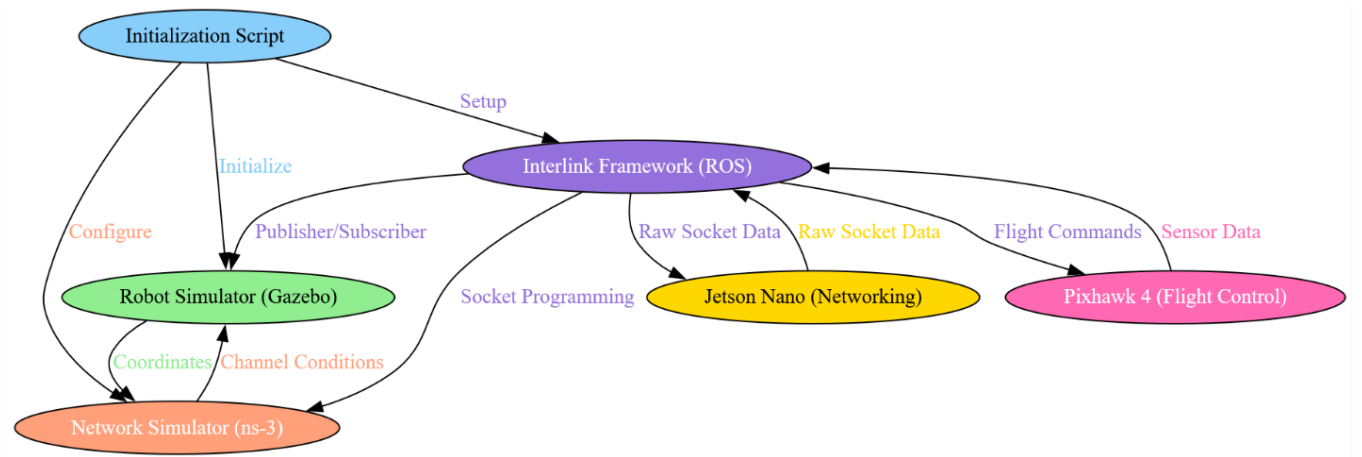


Figure 2: FANS Architecture

Table 1: Comparison of Network Simulation Stacks

Stack	Simulator Integration	Network Mode	Key Strengths	Key Limitations
FlyNetSim [2]	ns-3 + ArduPilot	Infrastructure	Realistic Flight Dynamics	Single point of failure (AP dependency)
CORNET [1]	ROS/Gazebo + ns-3	Infrastructure	Joint CPS Modelling	No Ad-hoc routing support
FANS [3]	ROS/Gazebo + ns-3	Ad-hoc	Decentralized Routing, Swarm Algorithms	Prior work lacked hardware validation
FANS Extended	FANS + Hardware	Ad-hoc	Sim-to-Real Validation, Scalability	Limited to small scale testbed (3 UAVs)

Forwarder Node: Listens on wlan0 and passes packets using raw sockets (promiscuous mode). Uses Realtime- Simulator to synchronize ns-3 time with hardware time.

Receiver Node: The packet arrival timestamps are recorded, and the end-to-end delay is calculated.

This hardware testbed is validated in a three-drone line formation, confirming decentralized ad-hoc communication feasibility under operational conditions. FANS Extended enhances the original platform through hardware emulation for physical validation. UAV Hardware Testbed for FANS Extended is depicted in Figure 1. Distinctive characteristics of the platform include:

- **Decentralized Ad-hoc Communication:** Improves swarm resilience by eliminating the need for centralized access points in FlyNetSim and CORNET.
- **Simulation-to-Reality Validation:** Deployment on Jetson Nano/Pixhawk 4 hardware indicates performance disparities such as increased delays and lower packet delivery ratios.
- **Operational Bottleneck Identification:** Identifies real-world constraints such as wireless interference, buffer limitations, and clock drift to aid in protocol improvement.

3 System Architecture

The FANS (Flying Ad-hoc Network Simulator) platform is intended to handle the issues of simulating and imitating Flying Ad-hoc Networks (FANETs) by combining network protocols, UAV dynamics, and hardware components into a single framework. The FANS

architecture, shown in Figure 2, is made up of four primary components: the network simulator (ns-3), the robot simulator (Gazebo), the interlink framework (ROS), and the startup script. A thorough overview of the system architecture is provided below, which includes hardware upgrades built for real-world emulation.

3.1 Initialization Script

The initialization script serves as the entry point for configuring the FANET environment. It defines:

- (1) Number of UAV nodes in the swarm (e.g., 3 drones in a straight-line formation)
- (2) Initial flight parameters for each UAV, including position, velocity, and altitude
- (3) Network configuration (e.g., IP addresses, MAC addresses, and communication ports).

This script uses the network simulator (ns-3) and the robot simulator (Gazebo) to create virtual UAV instances. For hardware emulation, it also configures embedded platforms (Jetson Nano + Pixhawk 4) by assigning IP addresses, enabling promiscuous mode on wireless interfaces, and granting root access for raw socket operations.

3.2 Robot Simulator (Gazebo)

Gazebo simulates the physical and kinematic behavior of UAVs, including:

- Mobility models (e.g., line formation, obstacle avoidance).
- Environmental interactions (e.g., wind resistance, inertia).

- Sensor emulation (e.g., GPS, IMU, downward-facing cameras for land-survey applications). In hardware emulation, Gazebo is replaced by Pixhawk 4 flight controllers for real-world UAV control, while ROS bridges the gap between simulated and hardware-based drone dynamics.

3.3 Network Simulator (ns-3)

Network Simulator (ns-3): ns-3 handles the networking aspects of FANETs, including:

- Ad-hoc WiFi communication (direct node-to-node links, avoiding centralized access points).
- Channel modeling (e.g., interference, path loss).
- Protocol execution (e.g., UDP packet transmission, congestion control).

The EmuFdNetDeviceHelper is used to attach simulated nodes to physical wireless interfaces (e.g., wlan0) for hardware emulation in ns-3. This makes it possible for real-time packet transmission and reception over actual hardware as well as the ability to model channel conditions.

3.4 Interlink Framework (ROS)

The Robot Operating System (ROS) acts as the middleware connecting the network and robot simulators. Key enhancements for hardware emulation include:

- (1) MAVROS Nodes: Interface with Pixhawk 4 flight controllers to relay UAV states (e.g., position, flight mode) to ns-3.
- (2) PCI Nodes: Manage message exchange between UAVs, ensuring synchronization of network and mobility data.
- (3) Planner: Maintains a centralized database of all published/subscribed messages (e.g., updated coordinates, network metrics).
- (4) Real-Time Synchronization: Uses RealtimeSimulatorImpl in ns-3 to align simulation time with wall-clock time, which is critical for hardware testing.

3.5 Hardware Integration

The hardware testbed extends FANS to embedded platforms, enabling real-world validation:

- (1) Jetson Nano: Acts as the compute module for executing ns-3 and ROS, handling packet forwarding via raw sockets.
- (2) Pixhawk 4: Manages low-level flight control, receiving commands from ROS and transmitting sensor data (e.g., GPS, IMU).
- (3) Data flow:
 - From Hardware to Simulation: Sensor data (e.g., UAV positions) is fed into ns-3 via ROS for dynamic channel updates.
 - From Simulation to Hardware: Network decisions (e.g., route updates) are sent to Pixhawk 4 for real-world UAV movement.

4 Implementation Methodology

The FANS (Flying Ad-hoc Network Simulator) platform provides interaction between network simulation, robot simulation, and

hardware components via a structured data exchange. This workflow is broken down into three phases: initialization, simulation loop, and hardware loop, with each phase aiming to synchronize UAV behavior and network protocols between virtual and physical environments.

The workflow starts with the **Initialization Script**, which serves as the foundation for configuring simulation and hardware emulation. For simulation, the script interacts with Gazebo and ns-3 to launch virtual UAV instances with the necessary parameters. In hardware emulation, it also configures embedded systems (Jetson Nano boards) via:

- (1) Setting static IP addresses for wireless interfaces (e.g., wlan0).
- (2) Enable promiscuous mode to record all incoming/outgoing packets.
- (3) Assign root privileges to raw socket creation program for low-level network access. This assures that the hardware setup matches the simulation environment, allowing for consistent validation of FANET protocols.

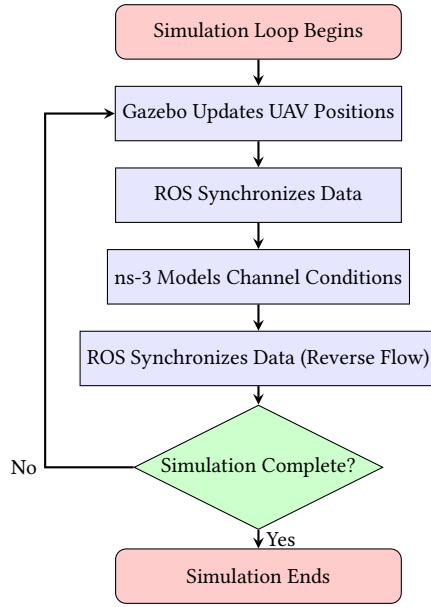
The workflow shown in Figures 3a and 3b makes sure that FANS supports both simulation and hardware testing, hence offering a full platform for FANET development and implementation.

5 Results

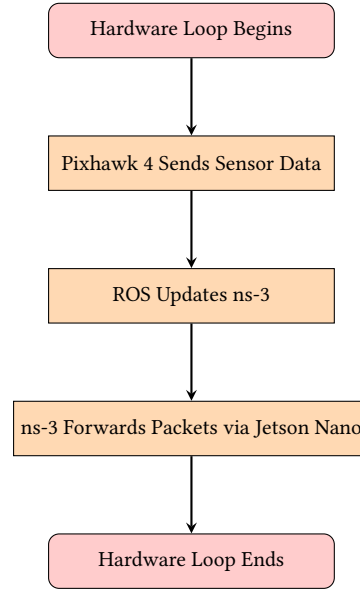
This section compares the performance of the FANS Extended platform in simulation and hardware environments, with a focus on delay, reliability, and the impact of real-world issues. Figure 4 shows the hardware testbed for FANS Extended. The hardware testbed uses IEEE 802.11g in ad-hoc mode to establish communication between nodes. In this configuration, Node 1 is a sender node, Node 2 is a forwarder node, and Node 3 is a receiver node. Packets sent between nodes include flight mode status, transmission and forwarding timestamps, and a unique sequence identification. UDP-based messaging demonstrated telemetry-like communication in a decentralized FANET environment. Figure 5 showcases terminal screen output of packet information on each individual node, and traces its evolution in the network.

To ensure timing consistency, the ns-3 simulation clock was synchronized with real-time hardware using the RealtimeSimulatorImpl interface. The end-to-end latency is calculated by comparing the transmission and receipt timestamps of each packet, as shown in Table 2. Because of ideal environmental simulations, forward (Node 1 to Node 2 to Node 3) and reverse (Node 3 to Node 2 to Node 1) links have consistently low latency, ranging from 0.4 to 0.45 milliseconds on average. In contrast, hardware test runs saw longer delays. Packets sent from Node 1 to Node 2 showed delays ranging from 94.03 to 121.41 milliseconds, with an average of 104.35 milliseconds. The second hop, from Node 2 to Node 3, was somewhat longer, with delays ranging from 72.79 to 116.82 milliseconds and an average of 100.87 milliseconds. This increase in time during hardware tests could be attributed to issues like operating system overhead, wireless channel contention, and socket buffering, all of which are avoided in simulation.

Packet delivery was also evaluated on three hardware runs. The simulation consistently demonstrated a perfect packet delivery ratio (PDR) of 100% across all links, as expected under optimal conditions. At the time of hardware implementation, the link from Node 1 to



(a) Flowchart of the Simulation Loop



(b) Flowchart of the Hardware Loop

Figure 3: Flowcharts of the Simulation and Hardware Loop

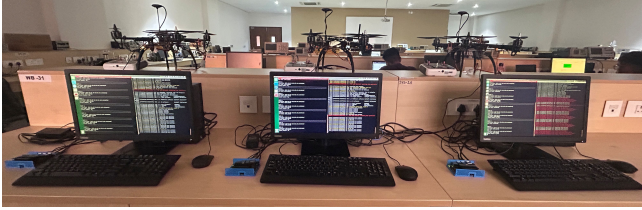


Figure 4: Hardware Testbed

Node 2 had one packet drop in Run 1 and two in Run 3, which resulted in an average PDR of 96.67%. The link between Node 2 and Node 3 achieved full delivery in two runs, but had a reduction in Run 3, resulting in an average PDR of 99.26%. While the overall system remained very dependable, transient wireless interference and timing instability resulted in occasional loss during hardware tests.

According to comparative analysis, simulation offers a baseline for ideal protocol behavior, but hardware testing discloses realistic limitations caused by real-world constraints. Environmental noise, radio interference, and CPU scheduling jitter on the embedded Jetson Nano platforms all contributed to non-deterministic delay changes. Furthermore, despite real-time synchronization systems, slight system clock mismatches impacted delay measurement precision, particularly in multi-node configurations. These findings emphasize the importance of accurate time alignment in distributed FANET deployments and argue that lightweight, resilient networking protocols are required to ensure good performance under constrained hardware constraints.

Overall, FANS Extended is able to replicate key simulation metrics in a physical environment with strong consistency, validating

the effectiveness of the platform while simultaneously revealing operational considerations that aid future protocol design.

6 Conclusion

FANS Extended brings together simulation and hardware components to provide a scalable and practical framework for analyzing FANET protocols. Using ns-3 and ROS on embedded hardware like the Jetson Nano and Pixhawk 4, we demonstrated that real-time decentralized UAV communication via ad hoc Wi-Fi is possible. Significant differences in delay and reliability were discovered between the simulated and real-world results, attributed to factors that are typically ignored in modeling systems, such as OS-level jitter, wireless interference, and buffer saturation.

Hardware-based validations highlight the need to take real-world networking parameters into account when designing Flying Ad Hoc Networks (FANETs). The findings of this research emphasize the need for accurate time synchronization in multinode installations. Although the initial implementation in this work is limited to a testbed of three UAVs, the architecture is modular and adaptable to larger swarms.

Future works will focus on expanding the system, improving synchronization using GPS-based timing, developing precise time-synchronization protocols, and introducing adaptive routing protocols that can respond to real-time network situations. As an outcome of this research, FANS Extended can be considered to be an important step toward building more robust, autonomous, and real-world deployable UAV networks.

Acknowledgments

This work is supported by the Science and Engineering Research Board (SERB-SRG/2019/001204), Government of India.

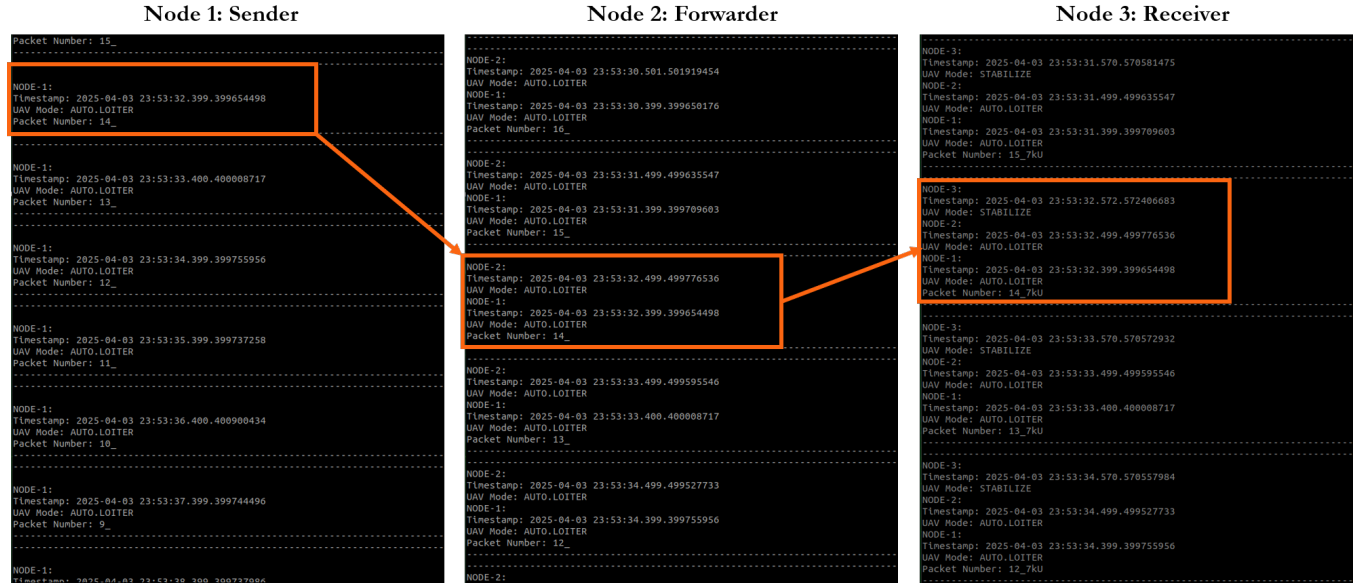


Figure 5: Packet Journey Captured through Terminal Output on Each Node

Table 2: Comparative Analysis of Simulation and Hardware Results

Link	Hardware Run 1		Hardware Run 2		Hardware Run 3		Hardware Avg.		Simulation Run	
	Delay (ms)	PDR (%)	Delay (ms)	PDR (%)	Delay (ms)	PDR (%)	Delay (ms)	PDR (%)	Delay (ms)	PDR (%)
N1 to N2	94.03	96.67	121.41	100.00	97.60	93.33	104.35	96.67	0.40	100.00
N2 to N3	113.01	100.00	72.79	100.00	116.82	97.78	100.87	99.26	0.45	100.00

Generative AI (ChatGPT) was utilized to generate sections of this Work, including text, tables, graphs, code, data, citations, etc.

References

- [1] Srikrishna Acharya, Amrutur Bharadwaj, Yogesh Simmhan, Aditya Gopalan, Parimal Parag, and Himanshu Tyagi. 2020. Cornet: A co-simulation middleware for robot networks. In *2020 International Conference on Communication Systems & Networks (COMSNETS)*. IEEE, 245–251.
- [2] Sabur Baidya, Zoheb Shaikh, and Marco Levorato. 2018. FlyNetSim: An Open Source Synchronized UAV Network Simulator Based on ns-3 and Ardupilot. In *Proceedings of the 21st ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWIM '18)* (Montreal, QC, Canada). Association for Computing Machinery, New York, NY, USA, 37–45. doi:10.1145/3242102.3242118
- [3] Sarang C. Dhongdi, Mohit P. Tahiliani, Ojit Mehta, Mihir Dharmadhikari, Vaibhav Agrawal, and Aditya Bidwai. 2022. FANS: Flying Ad-hoc Network Simulator. In *Proceedings of the 2022 Latin America Networking Conference (LANC 2022)* (Armenia, Colombia). Association for Computing Machinery, New York, NY, USA, 8. doi:10.1145/3545250.3560848
- [4] David Erdos, Abraham Erdos, and Steve E Watkins. 2013. An Experimental UAV System for Search and Rescue Challenge. *IEEE Aerospace and Electronic Systems Magazine* 28, 5 (2013), 32–37.
- [5] Thomas R Henderson, Mathieu Lacage, George F Riley, Craig Dowell, and Joseph Kopena. 2008. Network Simulations with the ns-3 Simulator. *SIGCOMM Demonstration* 14, 14 (2008), 527.
- [6] Nathan Koenig and Andrew Howard. 2004. Design and Use Paradigms for Gazebo, an Open-source Multi-robot Simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE Cat. No. 04CH37566 (Sendai, Japan), Vol. 3. IEEE, 2149–2154.
- [7] Mohammad Mozaffari, Walid Saad, Mehdi Bennis, Young-Han Nam, and M  rouane Debbah. 2019. A Tutorial on UAVs for Wireless Networks: Applications, Challenges, and Open Problems. *IEEE communications surveys & tutorials* 21, 3 (2019), 2334–2360.
- [8] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y Ng. 2009. ROS: an Open-source Robot Operating System. In *ICRA workshop on open source software*, Vol. 3. Kobe, Japan, 5.

A Appendix

The complete codebase used in this work is available at:
<https://github.com/Flying-Ad-Hoc-Networks/Extended-UDP>